

Assistente Inteligente de Estudos

Documentação Técnica Completa — Instalação, Configuração,
Referência de API, Operação e Solução de Problemas

Sumário

1. Sobre Esta Documentação	4
1.1 Público-alvo	4
1.2 Convenções	4
2. Visão Geral do Sistema	4
2.1 Componentes e Portas	5
3. Pré-requisitos	6
3.1 Software	6
3.2 Hardware (máquina local)	6
4. Estrutura do Projeto	6
5. Instalação	6
5.1 RAG Service (Python / FastAPI).....	7
5.2 IA / Proxy Service (Python / FastAPI)	7
5.3 Histórico Service (Python / FastAPI / PostgreSQL)	7
5.4 API Gateway (Node.js / Express)	7
5.5 MCP Service (TypeScript / Express)	7
5.6 Frontend (React / Vite).....	7
6. Configuração	7
6.1 Exemplos de arquivos .env	8
6.2 Parâmetros ajustáveis do Gateway	8
7. Ambiente de Nuvem — LLM no Google Colab	8
7.1 Passo 1 — Instalar dependências	8
7.2 Passo 2 — Subir o Ollama e baixar o modelo	8
7.3 Passo 3 — Abrir o túnel ngrok	9
8. Execução	9
9. Referência de API	9
9.1 API Gateway (porta 8000)	10
9.2 RAG Service (porta 8001).....	10
9.3 IA / Proxy Service (porta 8004).....	11
9.4 Histórico Service (porta 8003)	11
9.5 MCP Service (porta 8002)	12
10. Modelo de Dados	12
10.1 Tabela: mensagens	12

11. Fluxos de Uso	13
11.1 Enviar material e fazer perguntas	13
11.2 Pedir um resumo.....	13
11.3 Gerar questões	13
11.4 Perguntar sobre a identidade.....	13
12. Operação e Manutenção	13
13. Solução de Problemas (Troubleshooting)	14
14. Glossário	14

1. Sobre Esta Documentação

Este documento é o **manual técnico completo** do sistema **Assistente Inteligente de Estudos**, apelidado de **ARIA**. Ele reúne, em um único material, as informações necessárias para instalar, configurar, executar, operar e dar manutenção ao sistema, além de uma referência detalhada de todas as APIs expostas pelos microsserviços.

1.1 Público-alvo

- **Desenvolvedores** que vão instalar, modificar ou estender o sistema.
- **Operadores** responsáveis por executar e manter o ambiente em funcionamento.
- **Avaliadores** que desejam compreender a fundo o funcionamento da solução.

1.2 Convenções

- Comandos de terminal e trechos de código aparecem em `fonte monoespçada`.
- Variáveis a substituir pelo usuário são indicadas entre <colchetes angulares>.
- As portas indicadas (8000–8004, 5173) são os valores padrão usados no projeto.
- Os comandos assumem o sistema operacional **Windows** com **PowerShell**, o ambiente de desenvolvimento do projeto.

2. Visão Geral do Sistema

ARIA é um **tutor virtual acadêmico** que lê o material de aula de um estudante (PDF) e responde a perguntas sobre esse conteúdo, em português do Brasil. Quando a resposta não está no material, o sistema busca automaticamente na web. A solução adota uma arquitetura de **microsserviços**: cada responsabilidade é isolada em um serviço independente que se comunica com os demais por chamadas HTTP/JSON.

O sistema combina três técnicas centrais de Inteligência Artificial aplicada:

- **RAG (Retrieval-Augmented Generation)**: a resposta do modelo é ancorada em trechos recuperados do material, reduzindo alucinações.
- **Busca híbrida com reranking**: recuperação semântica (vetorial) + lexical (BM25), refinada por um *cross-encoder*.
- **LLM (Large Language Model)**: o modelo Qwen 2.5 14B gera as respostas em linguagem natural a partir do contexto recuperado.

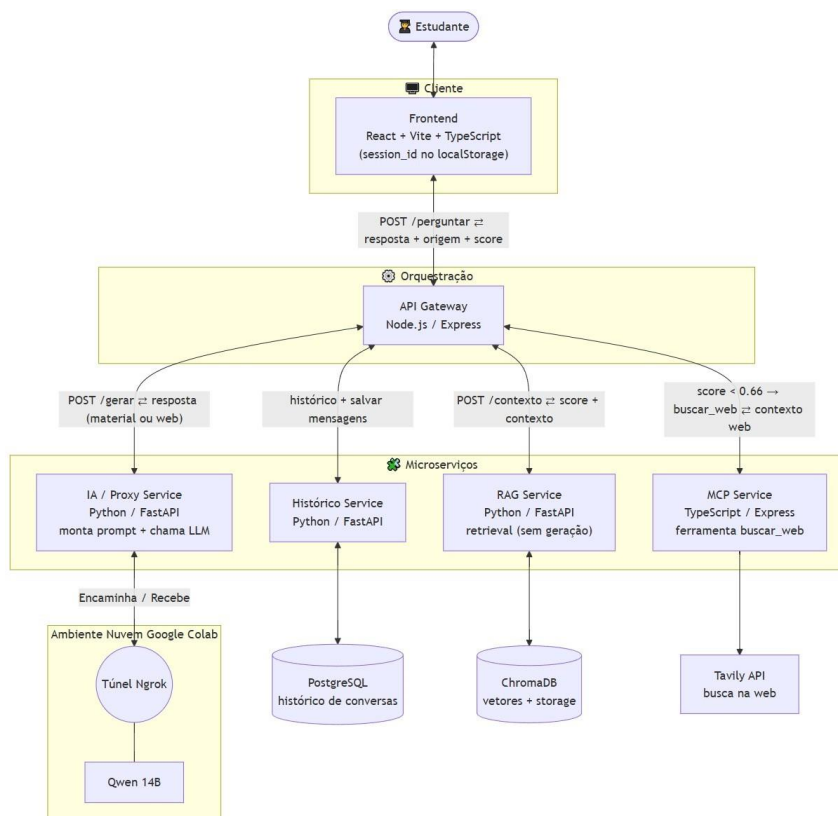


Figura 1 — Arquitetura de microserviços do sistema ARIA.

2.1 Componentes e Portas

Serviço	Stack	Porta	Papel
Frontend	React/Vite/TS	5173	Interface de chat e upload de PDF
API Gateway	Node/Express	8000	Orquestração e roteamento
RAG Service	Python/FastAPI	8001	Indexação e recuperação (retrieval)
MCP Service	TypeScript/Express	8002	Ferramenta de busca na web (Tavily)
Histórico Service	Python/FastAPI	8003	Memória da conversa (PostgreSQL)
IA / Proxy Service	Python/FastAPI	8004	Prompt + geração via LLM
LLM (Qwen 14B)	Ollama @ Colab	—	Modelo de linguagem (via ngrok)

3. Pré-requisitos

3.1 Software

Ferramenta	Versão recomendada	Usada por
Node.js + npm	18 LTS ou superior	API Gateway, MCP, Frontend
Python	3.12	RAG, IA/Proxy, Histórico
PostgreSQL	14 ou superior	Histórico Service
Conta Google (Colab)	—	Hospedagem do LLM (GPU gratuita)
Conta ngrok	túnel com domínio	Exposição do LLM à internet
Conta Tavily	chave de API	Busca na web (MCP Service)

3.2 Hardware (máquina local)

O RAG Service executa o modelo de embeddings na **GPU** e o reranker na **CPU**. A configuração de referência do projeto é uma **GPU NVIDIA com 4 GB de VRAM** (ex.: RTX 3050) e **16 GB de RAM**. É possível rodar apenas em CPU, alterando `device="cuda"` para `"cpu"` no RAG, ao custo de indexação mais lenta. O LLM em si **não** roda na máquina local — fica no Google Colab.

4. Estrutura do Projeto

A raiz do projeto contém uma pasta por microserviço, além dos artefatos de apoio (diagrama, material de exemplo e os scripts de geração de relatórios):

```
projeto_fase3/
├── frontend/                # React + Vite + TypeScript (interface de chat)
├── api_gateway/             # Node.js + Express (orquestrador, porta 8000)
│   └── server.js
├── rag_service/             # Python + FastAPI (retrieval, porta 8001)
│   └── main.py
│   ├── chroma_db/          # base vetorial ChromaDB (gerada)
│   └── storage/             # índice + nodes.pkl + docs.hash (gerados)
├── mcp_service/             # TypeScript + Express (busca web, porta 8002)
│   └── server.ts
├── historico_service/       # Python + FastAPI (PostgreSQL, porta 8003)
│   └── main.py
├── ia_service/              # Python + FastAPI (prompt + LLM, porta 8004)
│   └── main.py
├── documentos/              # material de aula (PDF) indexado pelo RAG
└── arquitetura_fase3.jpg
```

As pastas `chroma_db/` e `storage/` são criadas automaticamente pelo RAG na primeira indexação e podem ser apagadas para forçar uma reindexação do zero.

5. Instalação

Cada serviço é instalado de forma independente. Recomenda-se um terminal por serviço. As instruções abaixo partem da raiz do projeto.

5.1 RAG Service (Python / FastAPI)

Crie um ambiente virtual e instale as dependências de IA. O modelo de embeddings e o reranker são baixados do HuggingFace na primeira execução.

```
cd rag_service
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install fastapi uvicorn python-dotenv httpx cachetools pypdf
pip install llama-index-core llama-index-embeddings-huggingface \
    llama-index-llms-openai-like llama-index-readers-file \
    llama-index-vector-stores-chroma llama-index-retrievers-bm25
pip install chromadb sentence-transformers
# Para usar a GPU (CUDA), instale a build do PyTorch correspondente:
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

5.2 IA / Proxy Service (Python / FastAPI)

```
cd ia_service
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install fastapi uvicorn python-dotenv httpx openai \
    llama-index-core llama-index-llms-openai-like
```

5.3 Histórico Service (Python / FastAPI / PostgreSQL)

```
cd historico_service
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install fastapi uvicorn python-dotenv "sqlalchemy>=2" "psycopg[binary]"
```

Crie o banco de dados no PostgreSQL antes de iniciar o serviço (a tabela é criada automaticamente no primeiro startup):

```
CREATE DATABASE aria_historico;
```

5.4 API Gateway (Node.js / Express)

```
cd api_gateway
npm install
```

5.5 MCP Service (TypeScript / Express)

```
cd mcp_service
npm install
```

5.6 Frontend (React / Vite)

```
cd frontend
npm install
```

6. Configuração

As configurações sensíveis ficam em arquivos `.env`, um por serviço. A tabela a seguir resume todas as variáveis de ambiente do sistema.

Serviço	Variável	Descrição
ia_service	NGROK_URL	URL pública do LLM (Ollama) no Colab via ngrok
ia_service	LLM_TIMEOUT	Timeout (s) das chamadas ao LLM (padrão 600)

Serviço	Variável	Descrição
rag_service	NGROK_URL	Mesma URL do LLM (compatibilidade)
historico_service	DATABASE_URL	String de conexão do PostgreSQL
mcp_service	TAVILY_API_KEY	Chave da API Tavily (busca na web)

6.1 Exemplos de arquivos .env

ia_service/.env e rag_service/.env

```
NGROK_URL=https://<seu-subdominio>.ngrok-free.dev
```

historico_service/.env

```
DATABASE_URL=postgresql+psycopg://postgres:<senha>@localhost:5432/aria_historico
```

mcp_service/.env

```
TAVILY_API_KEY=tvly-<sua-chave-aqui>
```

Importante: sempre que o Colab for reiniciado e o túnel ngrok mudar, atualize a variável `NGROK_URL` nos serviços `ia_service` e `rag_service`.

6.2 Parâmetros ajustáveis do Gateway

O API Gateway aceita variáveis de ambiente opcionais para ajustar o roteamento e a memória sem alterar o código:

Variável	Padrão	Função
LIMIAR_RERANK	0.3	Limiar do rerank_score: $\geq$ → material; abaixo → web
LIMIAR_SCORE	0.66	Limiar de fallback (cosseno) quando não há rerank
HIST_LIMITE	6	Nº de mensagens anteriores usadas como memória
HIST_MAX_CHARS	600	Limite de caracteres por mensagem do histórico
GERAR_TIMEOUT	630000	Timeout (ms) das chamadas de geração ao IA Service

7. Ambiente de Nuvem — LLM no Google Colab

O modelo de linguagem (Qwen 2.5 14B) roda em uma instância do Google Colab com GPU, servido pelo **Ollama** e exposto à internet por um **túnel ngrok**. Execute as células do notebook na ordem abaixo.

7.1 Passo 1 — Instalar dependências

```
!apt-get install -y zstd lshw pciutils -q
!curl -fsSL https://ollama.com/install.sh | sh
!pip install pyngrok -q
```

7.2 Passo 2 — Subir o Ollama e baixar o modelo

A ordem é essencial: o servidor Ollama deve subir **antes** do `pull` do modelo.

```
import os, subprocess, time
os.system("OLLAMA_HOST=0.0.0.0 CUDA_VISIBLE_DEVICES=0 ollama serve > /tmp/ollama.log 2>&1 &")
time.sleep(10)
```



```
subprocess.run(["ollama", "pull", "qwen2.5:14b"], timeout=600)
```

7.3 Passo 3 — Abrir o túnel ngrok

```
import os, subprocess, time
os.makedirs("/root/.config/ngrok", exist_ok=True)
config = '''version: "3"
agent:
  authtoken: <SEU_AUTHTOKEN_NGROK>
tunnels:
  ollama:
    proto: http
    addr: 11434
    domain: <seu-subdominio>.ngrok-free.dev
    schemes: [https]'''
open("/root/.config/ngrok/ngrok.yml", "w").write(config)
subprocess.Popen(["ngrok", "start", "ollama", "--log", "/tmp/ngrok.log"])
```

Atenção: não defina `OLLAMA_CONTEXT_LENGTH=8192` no Colab — isso joga o modelo para a CPU e o torna lentíssimo. A janela de contexto do modelo é controlada pela aplicação (8.192 tokens no IA Service).

8. Execução

Com o LLM no ar (Colab + ngrok) e o PostgreSQL ativo, inicie os serviços locais — preferencialmente cada um em seu próprio terminal. A ordem recomendada coloca primeiro os serviços de retrieval e dados e, por último, o Gateway e o Frontend.

#	Serviço	Comando	Endereço
1	RAG Service	uvicorn main:app --port 8001	127.0.0.1:8001
2	Histórico	uvicorn main:app --port 8003	127.0.0.1:8003
3	IA / Proxy	uvicorn main:app --port 8004	127.0.0.1:8004
4	MCP Service	npm run start:ts	127.0.0.1:8002
5	API Gateway	npm start	127.0.0.1:8000
6	Frontend	npm run dev	127.0.0.1:5173

Os serviços Python também podem ser iniciados diretamente com `python main.py` (cada arquivo já chama o uvicorn na porta correta). Abra o navegador em `http://127.0.0.1:5173` para usar a aplicação.

Verificação rápida de saúde — cada serviço expõe um endpoint de status:

```
curl http://127.0.0.1:8001/health # RAG
curl http://127.0.0.1:8003/health # Histórico
curl http://127.0.0.1:8004/health # IA / Proxy (checa o LLM no Colab)
curl http://127.0.0.1:8002/health # MCP (checa a chave Tavily)
curl http://127.0.0.1:8000/      # Gateway (mostra os limiares ativos)
```

9. Referência de API

Esta seção documenta todos os endpoints expostos pelos microserviços. O **frontend consome exclusivamente o API Gateway**; os demais endpoints são internos (chamados entre serviços) e estão documentados para fins de manutenção e teste.

9.1 API Gateway (porta 8000)

POST /perguntar

Endpoint principal. Recebe a pergunta do estudante, orquestra recuperação/roteamento/geração e devolve a resposta com sua origem.

Corpo da requisição

```
{
  "pergunta": "O que é custo de oportunidade?",
  "session_id": "alb2c3d4-..."
}
```

Resposta (200 OK)

```
{
  "resposta": "***Custo de oportunidade** é ...",
  "origem": "material",
  "score": 0.78,
  "session_id": "alb2c3d4-..."
}
```

O campo `origem` assume os valores `material`, `web` ou `identidade`. Em caso de LLM indisponível, retorna **503** com o campo `erro` acionável.

POST /upload

Recebe um arquivo PDF (`multipart/form-data`, campo `arquivo`) e o repassa ao RAG para indexação. Aplica a política “um livro por vez”. Pode levar alguns minutos para PDFs grandes.

Resposta (200 OK)

```
{
  "mensagem": "'apostila.pdf' indexado com sucesso.",
  "chunks": 1240
}
```

9.2 RAG Service (porta 8001)

POST /contexto

Recupera trechos relevantes do material e devolve scores calibrados que orientam o roteamento. Pedidos de resumo/questões usam um caminho de amostra ampla.

Corpo / Resposta

```
// requisição
{ "texto": "custo de oportunidade" }

// resposta
{
  "score_max": 0.78,
  "rerank_score": 0.41,
  "trechos": ["...", "..."],
  "contexto": "...\\n\\n---\\n\\n..."
}
```

POST /upload

Recebe o PDF (campo `arquivo`), aplica a política “um livro por vez”, salva o arquivo e reindexa.

POST /debug

Diagnóstico: retorna os nós recuperados com rank, score, arquivo, página e trecho. Útil para calibrar limiares.

GET /ficha

Devolve título e autor do PDF indexado (usados como âncora temática na busca web). Retorna {} quando não há material.

POST /reindexar

Força a reconstrução completa do índice a partir do PDF presente na pasta `documentos/`.

DELETE /cache

Limpa o cache de respostas em memória (TTL).

GET /health

Status do serviço: se o índice está carregado e o número de *chunks* disponíveis.

9.3 IA / Proxy Service (porta 8004)

POST /gerar

Monta o prompt (com histórico e contexto) e gera a resposta via LLM. É chamado pelo Gateway.

Corpo da requisição

```
{
  "pergunta": "Resuma o material",
  "contexto": "trechos do PDF...",
  "fonte": "material",           // material | web | identidade
  "historico": "Estudante: ...\nARIA: ...",
  "reforcar_identidade": false
}
```

Resposta (200 OK)

```
{
  "resposta": "...",
  "fonte": "material"
}
```

GET /health

Verifica a conexão com o LLM no Colab/ngrok (`llm_online`).

9.4 Histórico Service (porta 8003)

POST /mensagens

Salva uma mensagem da conversa.

```
{
  "session_id": "alb2c3d4-...",
  "papel": "usuario",           // usuario | assistente
}
```

```
"conteudo": "texto da mensagem"
}
```

GET `/historico/{session_id}?limite=N`

Retorna as últimas N mensagens da sessão (padrão 10), em ordem cronológica.

DELETE `/historico/{session_id}`

Apaga todo o histórico de uma sessão.

GET `/health`

Verifica a conexão com o PostgreSQL.

9.5 MCP Service (porta 8002)

GET `/mcp/tools`

Lista as ferramentas externas disponíveis (descoberta MCP). Atualmente: `buscar_web`.

POST `/mcp/tools/call`

Executa uma ferramenta MCP. Para a busca na web:

```
// requisição
{ "tool": "buscar_web", "arguments": { "query": "..." } }

// resposta
{ "content": [ { "type": "text", "text": "resultados..." } ] }
```

GET `/health`

Status do serviço e se a chave Tavily está configurada.

10. Modelo de Dados

O Histórico Service persiste as conversas em PostgreSQL, no modelo normalizado “**uma linha por mensagem**”. A tabela é criada automaticamente no primeiro startup.

10.1 Tabela: mensagens

Coluna	Tipo	Descrição
id	INTEGER (PK, auto)	Identificador da mensagem
session_id	VARCHAR(128), indexado	Identificador da sessão/conversa
papel	VARCHAR(20)	'usuario' ou 'assistente'
conteudo	TEXT	Texto da mensagem
criado_em	TIMESTAMP (UTC)	Momento de criação

11. Fluxos de Uso

11.1 Enviar material e fazer perguntas

- 1 No frontend, clique no ícone de anexo (■) e selecione um PDF. Aguarde a confirmação de indexação.
- 2 Digite uma pergunta sobre o conteúdo e envie. A ARIA responde com base no material.
- 3 Se a pergunta não estiver no PDF, a resposta vem da web — e a ARIA avisa essa origem.

11.2 Pedir um resumo

Mensagens como “faça um resumo do material” acionam um caminho dedicado: o RAG monta uma **amostra ampla** do documento (início → meio → fim) e força a rota do material, garantindo cobertura de todo o conteúdo, não apenas do começo.

11.3 Gerar questões

Pedidos como “elabore 10 questões de múltipla escolha” injetam instruções de formato e anti-repetição. É possível pedir **sem gabarito** (“sem as respostas”) e escolher o tipo (abertas, fechadas ou mistas).

11.4 Perguntar sobre a identidade

Perguntas como “qual é o seu nome?” são respondidas de forma determinística — a assistente sempre se identifica como **ARIA**, sem depender do modelo nem do conteúdo do PDF.

12. Operação e Manutenção

Tarefa	Como fazer
Trocar o material indexado	Fazer upload de um novo PDF (substitui o anterior automaticamente)
Reindexar do zero	Apagar rag_service/storage e rag_service/chroma_db e reiniciar o RAG
Limpar o cache de respostas	DELETE http://127.0.0.1:8001/cache
Limpar o histórico de uma sessão	DELETE http://127.0.0.1:8003/historico/{session_id}
Atualizar o LLM após reiniciar o Colab	Editar NGROK_URL em ia_service/.env e rag_service/.env e reiniciar os dois
Calibrar o roteamento material × web	Ajustar LIMAR_RERANK (e LIMAR_SCORE) via variável de ambiente do Gateway

13. Solução de Problemas (Troubleshooting)

Sintoma	Causa provável	Solução
Resposta “A IA está offline” / erro 503	Colab caiu ou o túnel ngrok expirou	Reabrir o Colab, rodar ollama serve + ngrok e atualizar NGROK_URL
Erro 403 ao gerar	Túnel ngrok expirou	Reiniciar o ngrok no Colab e atualizar NGROK_URL
Modelo responde muito devagar / na CPU	OLLAMA_CONTEXT_LENGTH definido no Colab	Remover a variável; deixar a janela ser controlada pela aplicação
Busca web não funciona	TAVILY_API_KEY ausente	Configurar a chave em mcp_service/.env
Histórico indisponível	PostgreSQL fora do ar ou DATABASE_URL incorreta	Conferir o serviço do Postgres e a string de conexão (a conversa segue sem memória)
Erro de VRAM / OOM na indexação	GPU de 4 GB saturada	Reduzir embed_batch_size/TAMANHO_LOTE no RAG ou usar device='cpu'
Assuntos de livros diferentes se misturam	Índice antigo não foi limpo	Garantir um único PDF na pasta; o RAG limpa a coleção ao detectar mudança
Resumo “lê só o começo” do livro	Consulta genérica recuperou poucos trechos	Usar termos de resumo (já tratado pelo caminho de amostra ampla)

14. Glossário

Termo	Significado
RAG	Retrieval-Augmented Generation — geração ancorada em trechos recuperados
Embedding	Representação vetorial de um texto, que captura seu significado
Chunk	Fragmento do documento, unidade de indexação e recuperação
BM25	Algoritmo clássico de recuperação por relevância lexical (palavras)
Reranking	Reordenação dos candidatos por um modelo mais preciso (cross-encoder)
Cross-encoder	Modelo que avalia conjuntamente pergunta e trecho, dando relevância calibrada
LLM	Large Language Model — modelo de linguagem que gera o texto da resposta
MCP	Model Context Protocol — padrão para expor ferramentas externas ao modelo
Grounding	Ancoragem da resposta no material, para evitar informações inventadas
Ficha do documento	Chunk sintético com título/autor do PDF, para responder sobre o material
ngrok	Túnel que expõe um serviço local (o LLM no Colab) à internet

— Fim da documentação —